

Aula 5 — KMP: Kotlin Multiplatform + BFF

"Uma base de código, três plataformas"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC

Recap da Aula 4

- PWA: Web App Manifest + Service Worker lifecycle
- Cache strategies: CacheFirst, NetworkFirst, StaleWhileRevalidate
- Workbox: abstração declarativa de SW
- Background Sync + IndexedDB
- **Atividade 7** (PWA + TMDB) — entregue?

Agenda da aula

Bloco 1 (90min) — KMP: código compartilhado entre Android, iOS e Web

→ KotlinProject · setup toolchain · arquitetura shared · Ktor + TMDB · `expect` / `actual`

Intervalo (30min)

Bloco 2 (60min) — BFF/GraphQL + Projeto Final

→ Demo CineHub · REST vs GraphQL · Atividade 6 ao vivo · intro Projeto Final

Objetivos de aprendizagem

- Compartilhar lógica de negócio entre Android, iOS e Web com Kotlin
- Integrar HTTP com Ktor + `kotlinx.serialization` + TMDB API
- Entender `expect` / `actual` para código platform-specific
- Entender padrão Backend-for-Frontend — um servidor, múltiplos clientes
- Comparar REST (KMP/PWA) vs GraphQL (capstone CineHub)

KotlinProject — como foi criado

Projeto gerado pelo **KMP wizard oficial** da JetBrains:

kmp.jetbrains.com

Config usada: Android ✓ · iOS ✓ · iOS UI = **Compose** · Include Tests ✓

Essa URL reproduz exatamente o starter. Qualquer aluno pode gerar o mesmo projeto do zero no wizard — é o ponto de partida oficial pra projetos KMP novos.

O que vamos rodar hoje — KotlinProject

Três apps, um módulo compartilhado (`shared` — Compose Multiplatform):

App	Módulo	Plataforma
<code>androidApp</code>	<code>:androidApp</code>	Android
<code>iosApp</code>	<code>iosApp/</code>	iOS (Xcode project)
<code>webApp</code>	<code>:webApp</code>	Web (JS / Wasm)

Targets declarados em `shared/build.gradle.kts`:

`androidTarget`, `iosArm64`, `iosSimulatorArm64`, `js`, `wasmJs`

Toolchain verificado — versões exatas

Ferramenta	Versão
Android Studio	Quail 2026.1.1 Patch 2
Gradle wrapper	9.1.0
Kotlin	2.4.0
AGP	9.0.1
Compose Multiplatform	1.11.1
JDK (terminal)	OpenJDK 17.0.15
JDK (Studio)	bundled OpenJDK 21.0.10
Xcode	26.1.1

Projeto **não carrega** no Studio < Quail 2026.1.1. Atualizar se necessário.

Setup — pré-requisitos

```
# 1. Verificar Xcode CLI  
xcode-select -p  
# → /Applications/Xcode.app/Contents/Developer ✓
```

- **macOS** + Homebrew (/opt/homebrew/bin/brew)
- **Android Studio Quail 2026.1.1+** — versão exata importa
- **Xcode 26.x** com command-line tools selecionados
- `local.properties` criado automaticamente pelo Studio:

```
sdk.dir=/Users/<seu-user>/Library/Android/sdk
```

| Não commitar — é caminho de máquina.

Setup — plugin KMP no Android Studio

A parte mais traiçoeira. A configuração `iosApp` só aparece depois que o plugin carrega.

Passo a passo:

1. **Settings** → **Plugins** → **Marketplace** → buscar **Kotlin Multiplatform** (JetBrains s.r.o.) → Install
2. Se aparecer erro **Requires plugin 'com.intellij.marketplace' to be installed** + badge vermelho:
 - Marketplace → buscar **JetBrains Marketplace Licensing** → Install
3. Reiniciar o Studio
4. Dropdown de run config agora mostra `iosApp` ao lado de `androidApp` ✓

`kdoctor` pode reportar "KMM plugin not installed" — é falso positivo (procura o id legado 14936). Ignorar.

Setup — iOS Simulator (se não existir)

```
# Checar devices disponíveis
xcrun simctl list devices available

# Se a lista estiver vazia sob um runtime, criar:
xcrun simctl create "iPhone 16" \
    com.apple.CoreSimulator.SimDeviceType.iPhone-16 \
    com.apple.CoreSimulator.SimRuntime.iOS-26-1
```

Sintoma sem device: kdoctor avisa *"Couldn't find available Apple Simulators"*.

Verificar ambiente — kdoctor

```
brew install kdoctor  
kdoctor           # ou /opt/homebrew/bin/kdoctor
```

Esperado: OS ✓ · Java ✓ · Xcode ✓

2 warnings seguros para ignorar:

- *"Android Studio: KMM plugin not installed"* — kdoctor 1.1.0 checa id legado; nosso plugin está ok
- *"CocoaPods not configured"* — este projeto não tem `Podfile` ; só necessário com libs CocoaPods

Rodando os 3 targets

Android

- Dropdown → `androidApp` + `emulador/device` → 

iOS

- Dropdown → `iosApp` + `iPhone 16` → 
- Fallback (sem plugin):

```
open iosApp/iosApp.xcodeproj    # Run pelo Xcode
```

Web

```
./gradlew :webApp:jsBrowserDevelopmentRun    # JS  
./gradlew :webApp:wasmJsBrowserDevelopmentRun    # Wasm
```

Gotchas do KotlinProject

- **Apple Silicon:** target é `iosSimulatorArm64`. `iosX64` **não** é declarado → simuladores Intel não buildam.
- **Config cache com comportamento estranho:**

```
./gradlew --no-configuration-cache <task>
```

- **Nunca commitar** `local.properties` — caminho do SDK é específico de máquina.

Kotlin Multiplatform (KMP) — Stack & Forças

Stack:

- **Kotlin** linguagem em Android, iOS via Kotlin/Native, JS, JVM, Linux, Windows
- **expect / actual** — contratos compartilhados, implementações por plataforma
- **Compose Multiplatform** — UI compartilhada (em maturação)
- **Ktor** — HTTP client
- **kotlinx.serialization** — JSON

Forças:

- Compartilha **regra de negócio**, não UI (na maioria dos casos)
- Excelente integração com Android nativo
- iOS via Swift interop

Kotlin Multiplatform (KMP) — Fraquezas

- Talent pool ainda restrito
- iOS interop com debugging chato
- Compose Multiplatform ainda jovem em iOS

Kotlin — sintaxe em 1 slide

```
val nome = "Ana"           // imutável (≈ const)
var contador = 0           // mutável
fun saudar(n: String) = println("Oi, $n")    // função

data class Filme(val titulo: String, val nota: Double) // equals/copy/toString grátis

val apelido: String? = null // ? = pode ser null
val tam = apelido?.length ?: 0 // safe-call (?.) + elvis (?:)
listOf(1, 2, 3).filter { it > 1 } // lambda: it = o item
```

`val / var` · `data class` = seu modelo · `? / ?:` = null-safety · `{ it }` = lambda. É a linguagem do "shared" do KMP.

KMP — arquitetura (shared + targets)

commonMain – regra de negócio compartilhada

models, repositórios, casos de uso · `expect` declara o que muda por plataforma

▼ compila pra cada target ▼

androidMain

`actual` → OkHttp, APIs Android

iosMain

`actual` → URLSession, APIs iOS

Compartilha **lógica**, não UI. O que é específico de plataforma fica nos `actual`.

KMP — `expect` / `actual`

```
// commonMain
expect class HttpClient {
    suspend fun get(url: String): String
}

// androidMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // OkHttp implementation
    }
}

// iosMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // URLSession implementation
    }
}
```

`expect` declara o contrato. `actual` implementa por plataforma.

Hands-on guiado — Kotlin (Playground)

play.kotlinlang.org (zero instalação). Escreva a "shared logic" dos favoritos — a mesma do `favoritesStore` do app:

```
data class Movie(val id: Int, val title: String, val rating: Double) // 1. modelo

fun favorites(movies: List<Movie>, favIds: Set<Int>) = // 2. função pura
    movies.filter { it.id in favIds }

fun main() {
    val all = listOf(
        Movie(1, "Matrix", 8.7),
        Movie(2, "Inception", 8.8),
        Movie(3, "Tenet", 7.4),
    )
    val favs = favorites(all, setOf(1, 3)) // 3. rode!
    println(favs.map { it.title }) // → [Matrix, Tenet]
}
```

(1) `data class` = modelo · (2) função pura de filtro · (3) rode. Essa lógica o KMP compartilha entre Android e iOS — só o que toca plataforma (rede, sensores) vira `expect / actual`.

Estudos de caso — KMP

KMP:

- Square (Cash App)
- JetBrains (próprio Toolbox, Fleet)
- McDonald's mobile
- Philips

Intervalo — 30min

Volta às :____

Aproveita pra:

- Esticar perna
- Pegar token TMDb se não tiver: developer.themoviedb.org → API → Bearer
- Android Studio aberto com o projeto KMP carregado

Próximo bloco: BFF/GraphQL + Atividade 6.

BFF — Backend-for-Frontend

O problema sem BFF

- RN, KMP, Web cada um bate em endpoints REST diferentes
- Over-fetching (campos que não usa) e under-fetching (N+1 requests)
- Múltiplos times sincronizando formatos de resposta

Com BFF + GraphQL

- Um endpoint `/graphql` serve **todos os clientes**
- Cliente pede **só os campos que precisa**
- Schema = contrato único versionado

```
# Android pergunta isso:
query {
  popularMovies {
    results { title voteAverage }
  }
}

# PWA pergunta isso:
query {
  popularMovies {
    results { title posterPath overview }
  }
}
```

Mesma query shape, respostas diferentes. O BFF otimiza por cliente sem mudar o backend.

GraphQL na prática — o servidor CineHub

```
# No terminal — pasta compartilhado/capstone/server  
npm install && npm run seed && npm start  
# → CineHub GraphQL pronto em http://localhost:4000/
```

Abra <http://localhost:4000/> no browser — Playground Apollo Studio.

Queries disponíveis:

```
{ popularMovies(page: 1) { page results { id title voteAverage } } }  
  
{ searchMovies(query: "Matrix") { id title releaseDate } }  
  
{ movie(id: 550) { title overview } }
```

O CineHub tem dados de filmes clássicos no SQLite local. Sem TMDB token necessário — ótimo pra demo.

TMDB REST vs GraphQL — dois sabores do mesmo problema

	TMDB REST (exercícios)	CineHub GraphQL (demo)
Autenticação	Bearer token no header	JWT local (register/login)
Popular movies	GET /movie/popular	query { popularMovies }
Busca	GET /search/movie?query=x	query { searchMovies(query:"x") }
Over-fetching	Sim (resposta fixa)	Não (cliente escolhe campos)
Offline	Cache HTTP / SW	Normalização client-side

No KMP usamos Ktor + TMDB REST — simples, sem servidor local.

Na PWA usamos fetch + TMDB REST + Service Worker para offline.

No BFF real (capstone) tudo passa pelo GraphQL.

Atividade 6 — KMP: filmes populares (10 pts)

Stack: o mesmo KotlinProject do Bloco 1 (Android/iOS/Web) + Ktor + TMDB REST API

Já pronto (`shared/src/commonMain/kotlin/.../data/`): `Movie.kt` + `TmdbApi.kt` (busca real via Ktor). **O exercício é construir a UI** em `App.kt` — 3 TODOs:

```
TODO 1 estados: token ausente / carregando / erro / lista
TODO 2 MovieList — LazyColumn com cabeçalho + itens
TODO 3 MovieCard — poster placeholder + título + overview + nota/ano
```

Setup:

```
cd exercicios/06-kmp/pratica
cp local.properties.example local.properties
# adicione seu tmdb.token= no local.properties
# Sync Gradle → Run androidApp
```

Entrega: fork → PR tocando `exercicios/06-kmp/pratica/` · Prazo: canvas

Projeto Final — 5 temas, 3 frameworks, E2E de verdade

O CineHub acima foi só a **demo** de GraphQL desta aula — o Projeto Final é outro projeto: grupo escolhe 1 tema (PokeAPI, Rick and Morty, TheMealDB, TheCocktailDB ou Studio Ghibli) + 1 framework (Flutter, RN ou KMP) e completa 5 features pra passar 5 testes E2E (Maestro) rodando de verdade em CI.

Enunciado completo: `exercicios/projeto-final/enunciado.md`. Entrega: fork → PR tocando `exercicios/projeto-final/skeletons/<tema>/<framework>/pratica/`.

Leituras complementares

- **JetBrains** — *Kotlin Multiplatform Docs* (kotlinlang.org/docs/multiplatform)
- **Ktor** — *ktor.io/docs/getting-started-ktor-client.html*
- **Newman, S.** — *Building Microservices* 2ed, capítulo 4 (BFF)
- **Porcello & Banks** — *Learning GraphQL* (O'Reilly)

Obrigado!

✉ jackson.96@gmail.com

🔗 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

🐙 github.com/jacksonsmith

Bom trabalho final! 🚀